

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT on

ANALYSIS AND DESIGN OF ALGORITHMS AAT REPORT

Submitted by

Chirag C S (1BM4CS085)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “ANALYSIS AND DESIGN OF ALGORITHMS”– 23CS4PCADA carried out by Chirag C S (1BM24CS085), who is a Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2026. The Lab report has been approved as it satisfies the academic requirements in respect of an ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA) work prescribed for the said degree.

Mayanka Gupta
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	LeetCode exercises on Stacks, Queues, Circular Queues, Priority Queues.	01
2.	LeetCode exercises on DFS, BFS	04
3.	LeetCode exercises on Trees and Graphs	05
4.	Write program to obtain the Topological ordering of vertices in a given digraph	06
	Leet Code exercises on topological sorting.	08
5.	Implement Johnson Trotter algorithm to generate permutations.	09
6.	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	14
7.	Sort a given set of N integer elements using Quick Sort technique and compute its time taken	18
8.	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	21
9.	Implement 0/1 Knapsack problem using dynamic programming	26
	Leet Code exercise on Knapsack problem using dynamic programming.	29
10.	Implement All Pair Shortest paths problem using Floyd's algorithm.	30

	Leet Code exercise on Floyd's algorithm.	32
11.	Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.	33
	Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	36
12.	Implement fractional knapsack problem using Greedy technique.	40
13.	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	43
14.	Implement "N-Queens Problem" using Backtracking.	46

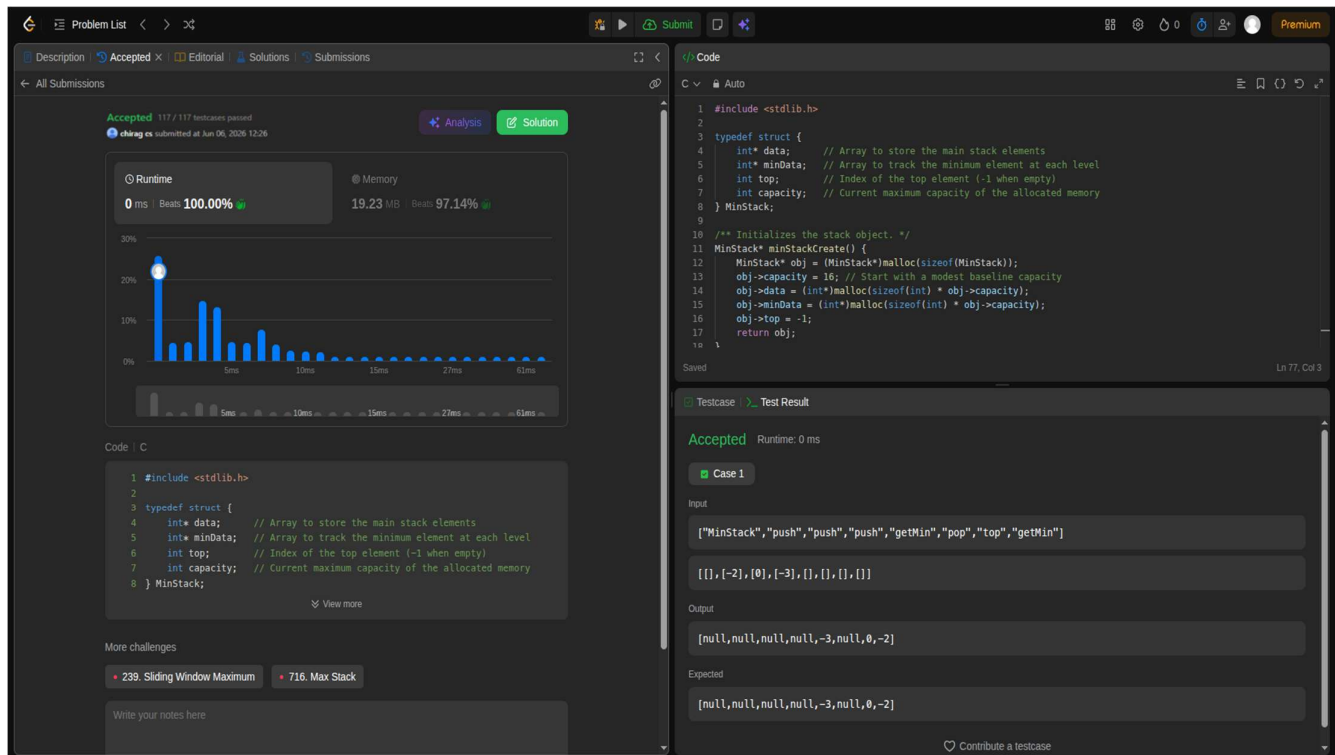
Program -1

Question:

LeetCode exercises on Stacks, Queues, Circular Queues, Priority Queues.

solution :

stacks :



Queues :

Accepted 117 / 117 testcases passed
 ching cs submitted at Jun 06, 2026 12:26

Runtime 34 ms | Beats 10.43%
Memory 42.28 MB | Beats 80.12%

Code | C

```
1 #include <stdlib.h>
2
3 #define MAX_CALLS 10000
4
5 typedef struct {
6     int* queue; // Array to store timestamps
7     int head; // Index pointing to the oldest valid timestamp
8     int tail; // Index pointing to where the next timestamp will be inserted
9 } RecentCounter;
10
11 /** Initializes the counter with zero recent requests. */
12 RecentCounter* recentCounterCreate() {
13     RecentCounter* obj = (RecentCounter*)malloc(sizeof(RecentCounter));
14     // Based on the constraints, at most 10^4 calls will be made to ping.
15     obj->queue = (int*)malloc(sizeof(int) * MAX_CALLS);
16     obj->head = 0;
17     obj->tail = 0;
18     return obj;
19 }
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Input

["RecentCounter","ping","ping","ping","ping"]

[[],[1],[100],[3001],[3002]]

Output

[null,1,2,3,3]

Expected

[null,1,2,3,3]

Circular Queues :

Accepted 117 / 117 testcases passed
 ching cs submitted at Jun 06, 2026 12:26

Runtime 6 ms | Beats 28.68%
Memory 18.92 MB | Beats 48.43%

Code | C

```
1 #include <stdbool.h>
2 #include <stdlib.h>
3
4 typedef struct {
5     int* data; // Array to store queue elements
6     int head; // Index of the front element
7     int tail; // Index of the rear element
8     int size; // Current number of elements in the queue
9     int capacity; // Maximum capacity of the queue (k)
10 } MyCircularQueue;
11
12 /** Initializes the object with the size of the queue to be k. */
13 MyCircularQueue* myCircularQueueCreate(int k) {
14     MyCircularQueue* obj = (MyCircularQueue*)malloc(sizeof(MyCircularQueue));
15     obj->data = (int*)malloc(sizeof(int) * k);
16     obj->head = 0;
17     obj->tail = -1; // Initialized to -1 so the first push lands at index 0
18     obj->size = 0;
19     obj->capacity = k;
20     return obj;
21 }
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Input

["MyCircularQueue","enQueue","enQueue","enQueue","enQueue","Rear","isFull","deQueue","enQueue","Rear"]

[[],[1],[2],[3],[4],[1],[1],[4],[1]]

Output

[null,true,true,true,false,3,true,true,true,4]

Expected

[null,true,true,true,false,3,true,true,true,4]

Priority Queues :

The screenshot displays a LeetCode submission interface. On the left, the 'Runtime' section shows a bar chart with a single bar at 0 ms, indicating it beats 100.00% of other submissions. The 'Memory' section shows 8.68 MB usage, beating 52.88% of others. The 'Code' section contains the following C++ code:

```

1 #include <stdlib.h>
2
3 // Helper function to maintain max-heap property downward
4 void heapifyDown(int* heap, int size, int index) {
5     int largest = index;
6     int left = 2 * index + 1;
7     int right = 2 * index + 2;
8
9     if (left < size && heap[left] > heap[largest]) {
10         largest = left;
11     }
12     if (right < size && heap[right] > heap[largest]) {
13         largest = right;
14     }
15
16     if (largest != index) {
17         int temp = heap[index];
18         heap[index] = heap[largest];
19         heap[largest] = temp;
20     }
21 }

```

On the right, the 'Test Result' section shows 'Accepted' with a runtime of 0 ms. The input is 'stones = [2,7,4,1,8,1]' and the output is '1'. The expected result is also '1'.

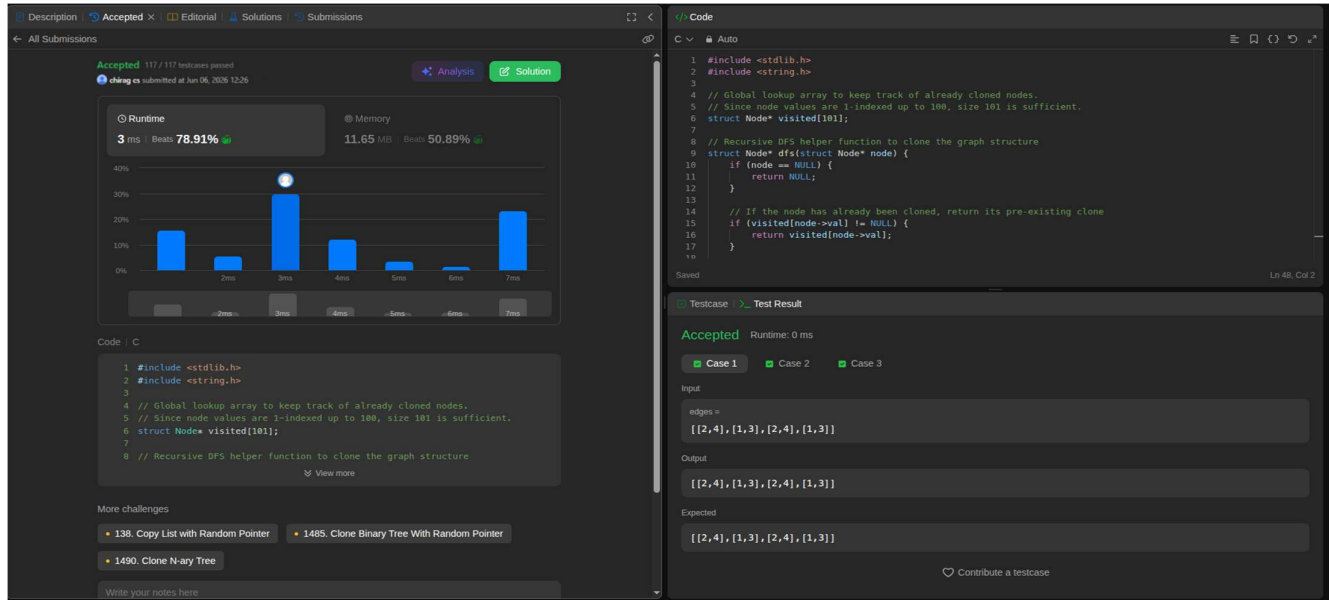
Program -2

Question:

LeetCode exercises on DFS, BFS

solution :

DFS :



BSF :

The screenshot shows a LeetCode submission for the problem "Binary Tree Level Order Traversal". The submission is accepted, with a runtime of 0 ms and memory usage of 12.34 MB. The code is written in C and implements a breadth-first search (BSF) algorithm to traverse the tree level by level. The test result shows that the solution passes all test cases, including Case 1, Case 2, and Case 3.

```

1 #include <stdlib.h>
2 #include <string.h>
3
4 // Helper function to calculate the max depth (height) of the tree
5 int getDepth(struct TreeNode* root) {
6     if (root == NULL) return 0;
7     int leftDepth = getDepth(root->left);
8     int rightDepth = getDepth(root->right);
9     return (leftDepth > rightDepth ? leftDepth : rightDepth) + 1;
10 }
11
12 int** levelOrderBottom(struct TreeNode* root, int* returnSize, int** returnColumnSizes) {
13     if (root == NULL) {
14         *returnSize = 0;
15         *returnColumnSizes = NULL;
16         return NULL;
17     }
18 }

```

Testcase: Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input: root = [3,9,20,null,null,15,7]

Output: [[15,7],[9,20],[3]]

Expected: [[15,7],[9,20],[3]]

Program -3**Question:**

LeetCode exercises on Trees and Graphs

solution :**Trees :**

The screenshot shows a LeetCode submission for the problem "Reverse Odd Levels of Binary Tree". The submission is accepted, with a runtime of 0 ms and memory usage of 9.26 MB. The code is written in C and implements a recursive algorithm to invert the tree at odd levels. The test result shows that the solution passes all test cases, including Case 1, Case 2, and Case 3.

```

1 #include <stdlib.h>
2
3 struct TreeNode* invertTree(struct TreeNode* root) {
4     // Base case: if the node is null, nothing to invert
5     if (root == NULL) {
6         return NULL;
7     }
8 }
9
10 // Recursively process the subtrees
11 struct TreeNode* leftInverted = invertTree(root->left);
12 struct TreeNode* rightInverted = invertTree(root->right);
13
14 // Swap the left and right children of the current node
15 root->left = rightInverted;
16 root->right = leftInverted;
17
18 // Return the root of the inverted tree
19 return root;

```

Testcase: Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input: root = [4,2,7,1,3,6,9]

Output: [4,7,2,9,6,3,1]

Expected: [4,7,2,9,6,3,1]

Graphs :

The screenshot displays a code editor with a C program for finding the minimum height tree. The program is titled "Accepted" and shows a runtime analysis graph. The graph plots runtime (ms) and memory (MB) against the number of vertices (n). The runtime is 20 ms and memory is 25.09 MB. The code is as follows:

```

1 #include <stdlib.h>
2 #include <string.h>
3
4 int* findMinHeightTrees(int n, int** edges, int edgesSize, int* edgesColSize, int* returnSize) {
5     // Edge case: 1 or 2 nodes are their own centroids
6     if (n <= 2) {
7         *returnSize = n;
8         int* result = (int*)malloc(sizeof(int) * n);
9         for (int i = 0; i < n; i++) {
10             result[i] = i;
11         }
12         return result;
13     }
14
15     // 1. Allocate Adjacency List dynamically
16     // To save space and avoid an O(N^2) matrix, we track degrees to mimic a dynamic list sizing
17     int* degrees = (int*)calloc(n, sizeof(int));
18     for (int i = 0; i < edgesSize; i++) {
19         for (int j = 0; j < edgesColSize[i]; j++) {
20             int u = edges[i][j];
21             int v = edges[i][j+1];
22             degrees[u]++;
23             degrees[v]++;
24         }
25     }
26
27     // 2. Find centroids (nodes with degree <= 1)
28     int* result = (int*)malloc(sizeof(int) * n);
29     int* visited = (int*)calloc(n, sizeof(int));
30     int* queue = (int*)malloc(sizeof(int) * n);
31     int front = 0, rear = 0;
32     for (int i = 0; i < n; i++) {
33         if (degrees[i] <= 1) {
34             queue[rear++] = i;
35             visited[i] = 1;
36         }
37     }
38
39     // 3. Remove leaves iteratively until 1 or 2 nodes remain
40     while (n > 2) {
41         n--;
42         for (int i = 0; i < edgesSize; i++) {
43             for (int j = 0; j < edgesColSize[i]; j++) {
44                 int u = edges[i][j];
45                 int v = edges[i][j+1];
46                 if (visited[u] || visited[v]) continue;
47                 degrees[u]--;
48                 degrees[v]--;
49                 if (degrees[u] <= 1) queue[rear++] = u;
50                 if (degrees[v] <= 1) queue[rear++] = v;
51                 visited[u] = 1;
52                 visited[v] = 1;
53             }
54         }
55     }
56
57     *returnSize = n;
58     for (int i = 0; i < n; i++) {
59         result[i] = queue[i];
60     }
61     return result;
62 }

```

The test case result section shows the following input and output:

```

Input:
n = 4
edges = [[1,0],[1,2],[1,3]]

Output:
[1]

Expected:
[1]

```

Program -4

Question:

Write a program to obtain the Topological ordering of vertices in a given digraph.

code :

```

#include <stdio.h>
int main()
{
    int n;
    int adj[20][20];
    int indegree[20] = {0};
    int visited[20] = {0};

    printf("Enter number of vertices: ");
    scanf("%d", &n);

```

```
printf("Enter adjacency matrix:\n");
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        scanf("%d", &adj[i][j]);
    }
}

for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        if (adj[i][j] == 1)
        {
            indegree[j]++;
        }
    }
}
printf("Topological Sorting Order:\n");
for (int count = 0; count < n; count++)
{
    for (int i = 0; i < n; i++)
    {
        if (indegree[i] == 0 && visited[i] == 0)
        {
            printf("%d ", i);
            visited[i] = 1;

            for (int j = 0; j < n; j++)
            {
                if (adj[i][j] == 1)
```

```

        {
            indegree[j]--;
        }
    }
}

return 0;
}

```

Result :

```

Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0
Topological Sorting Order:
0 1 2 3

```

Leet Code exercises on topological sorting.

The screenshot displays a LeetCode submission interface. On the left, the 'Runtime' section shows a bar chart with a peak at 5ms, indicating the submission is faster than 100% of other solutions. The 'Memory' section shows a usage of 24.70 MB, which is better than 90% of other solutions. The C code is shown in the center, implementing a DFS-based topological sort. On the right, the 'Testcase' section shows the submission is 'Accepted' with a runtime of 0 ms. The input is a graph with 6 nodes and 5 edges, and the output is the topological order [2, 4, 5, 6].

Runtime Performance:

- Runtime: 0 ms, Beats: 100.00%
- Memory: 24.70 MB, Beats: 90.00%

Code (C):

```
1 #include <stdlib.h>
2 #include <stdbool.h>
3
4 // DFS helper function to detect cycles and mark safe nodes
5 bool dfs(int node, int** graph, int* graphColSize, int* state) {
6     // If already visiting this node in current path, a cycle is detected (u
7     if (state[node] == 1) {
8         return false;
9     }
10    // If node is already visited and not in current path, it's a safe node
11    if (state[node] == 2) {
12        return true;
13    }
14    state[node] = 1;
15    for (int i = 0; i < graphColSize[node]; i++) {
16        if (!dfs(graph[node][i], graph, graphColSize, state)) {
17            return false;
18        }
19    }
20    state[node] = 2;
21    return true;
22 }
```

Testcase Results:

- Accepted (Runtime: 0 ms)
- Case 1: Input: graph = [[1,2],[2,3],[5],[0],[5],[1],[1]] Output: [2,4,5,6] Expected: [2,4,5,6]

Program -5**Question:**

Implement Johnson Trotter algorithm to generate permutations.

code :

```
#include <stdio.h>
#define LEFT -1
#define RIGHT 1

int getMobile(int a[], int dir[], int n)
{
    int mobile = 0;
    int mobile_index = -1;

    for (int i = 0; i < n; i++)
    {
        if (dir[a[i]] == LEFT && i != 0)
        {
            if (a[i] > a[i - 1] && a[i] > mobile)
            {
                mobile = a[i];
                mobile_index = i;
            }
        }
        if (dir[a[i]] == RIGHT && i != n - 1)
        {
            if (a[i] > a[i + 1] && a[i] > mobile)
            {
                mobile = a[i];
                mobile_index = i;
            }
        }
    }
}
```

```

    }
}

return mobile_index;
}

void printPermutation(int a[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

void johnsonTrotter(int n)
{
    int a[20], dir[21];
    for (int i = 0; i < n; i++)
    {
        a[i] = i + 1;
        dir[i + 1] = LEFT;
    }
    printPermutation(a, n);
    while (1)
    {
        int mobile_index = getMobile(a, dir, n);
        if (mobile_index == -1)
            break;
        int mobile = a[mobile_index];
        if (dir[mobile] == LEFT)
        {
            int temp = a[mobile_index];
            a[mobile_index] = a[mobile_index - 1];
            a[mobile_index - 1] = temp;
            mobile_index--;
        }
    }
}

```

```

    }
    else
    {
        int temp = a[mobile_index];
        a[mobile_index] = a[mobile_index + 1];
        a[mobile_index + 1] = temp;
        mobile_index++;
    }
    for (int i = 1; i <= n; i++)
    {
        if (i > mobile)
            dir[i] = -dir[i];
    }
    printPermutation(a, n);
}
}

int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    johnsonTrotter(n);
    return 0;
}

```


Result :

```
Enter number of elements: 4
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4
```

Program -6**Question:**

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

code :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];

    for(i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for(j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    i = 0;
    j = 0;
    k = l;

    while(i < n1 && j < n2) {
        if(L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
```

```

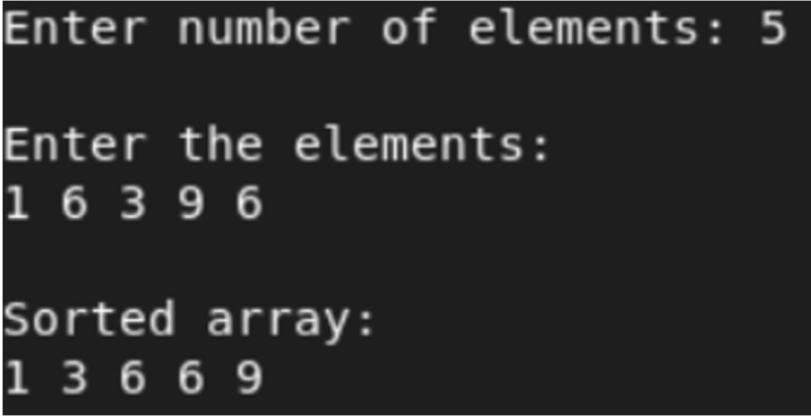
    }
    else {
        arr[k] = R[j];
        j++;
    }
    k++;
}
while(i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}
while(j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
}
void mergeSort(int arr[], int l, int r) {
    if(l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        merge(arr, l, m, r);
    }
}
int main() {
    int n, i;
    clock_t start, end;
    double cpu_time;
    printf("Enter number of elements: ");

```

```
scanf("%d", &n);
int arr[n];
printf("Enter %d elements:\n", n);
for(i = 0; i < n; i++)
{
    arr[i]=rand()%1000;
}
start = clock();
mergeSort(arr, 0, n - 1);
end = clock();
cpu_time = ((double)(end - start)*1000) / CLOCKS_PER_SEC;
printf("\nSorted array:\n");
for(i = 0; i < n; i++)
    printf("%d ", arr[i]);
printf("\n\nTime taken = %f seconds\n", cpu_time);
return 0;
}
```

Result :

A screenshot of a terminal window showing the execution of a C program. The text is as follows:

```
Enter number of elements: 5
Enter the elements:
1 6 3 9 6
Sorted array:
1 3 6 6 9
```

Program -7**Question:**

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

code :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1, j, temp;

    for(j = low; j < high; j++)
    {
        if(arr[j] <= pivot)
        {
            i++;
            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}
```

```

void quickSort(int arr[], int low, int high)
{
    if(low < high)
    {
        int pi = partition(arr, low, high);

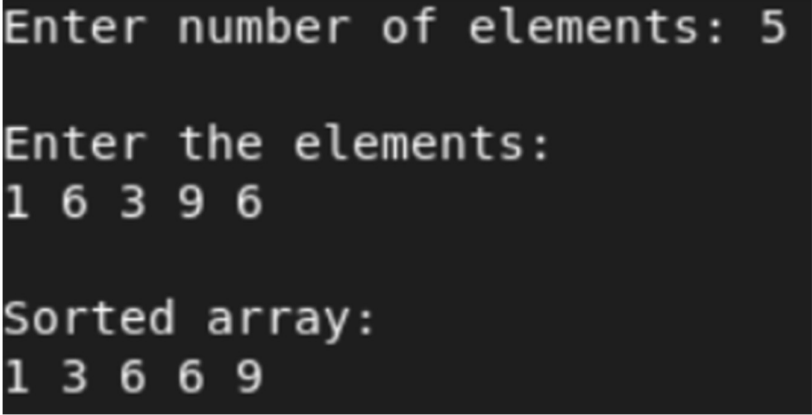
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main()
{
    int n, i;
    clock_t start, end;
    double cpu_time;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    srand(time(0)); // Random seed
    printf("Generated array:\n");
    for(i = 0; i < n; i++)
    {
        arr[i] = rand() % 1000;
        printf("%d ", arr[i]);
    }
    start = clock();
    quickSort(arr, 0, n - 1);
    end = clock();
    cpu_time = ((double)(end - start) * 1000) / CLOCKS_PER_SEC;
    printf("\n\nSorted array:\n");
    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);
}

```

```
printf("\n\nTime taken = %f ms\n", cpu_time);  
return 0;  
}
```

Result :



```
Enter number of elements: 5  
  
Enter the elements:  
1 6 3 9 6  
  
Sorted array:  
1 3 6 6 9
```

Program -8**Question:**

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

code :

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>

void heapcom(int a[], int n)
{
    int i, j, k, item;

    for(i=1; i<=n; i++)
    {
        item = a[i];
        j = i;
        k = j/2;

        while(k!=0 && item>a[k])
        {
            a[j] = a[k];
            j = k;
            k = j/2;
        }
        a[j] = item;
    }
}

void adjust(int a[], int n)
{
```



```

int item, i, j;
j = 1;
item = a[j];
i = 2*j;
while(i < n)
{
    if((i+1) < n)
    {
        if(a[i] < a[i+1])
            i++;
    }

    if(item < a[i])
    {
        a[j] = a[i];
        j = i;
        i = 2*j;
    }
    else
        break;
}
a[j] = item;
}

void heapsort(int a[], int n)
{
    int i, temp;
    heapcom(a, n);
    for(i=n; i>=1; i--)
    {
        temp = a[1];
        a[1] = a[i];
        a[i] = temp;
    }
}

```

```

        adjust(a, i);
    }
}

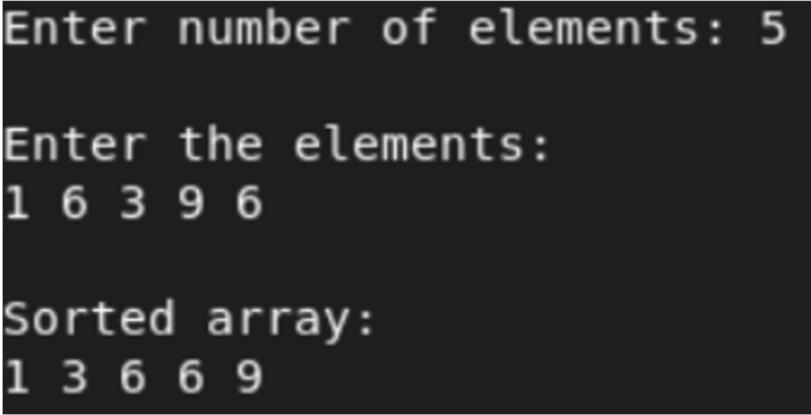
int main()
{
    int i, n;
    int a[100000];
    clock_t start, end;
    double cpu_time;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    srand(time(0));

    for(i=1; i<=n; i++)
    {
        a[i] = rand() % 1000;
    }
    printf("\nRandom elements are:\n");
    for(i=1; i<=n; i++)
    {
        printf("%d ", a[i]);
    }
    start = clock();
    heapsort(a, n);
    end = clock();
    cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\n\nSorted elements are:\n");
    for(i=1; i<=n; i++)
    {
        printf("%d ", a[i]);
    }
}

```

```
}  
printf("\n\nTime taken = %lf seconds\n", cpu_time);  
return 0;  
}
```

Result :



```
Enter number of elements: 5  
  
Enter the elements:  
1 6 3 9 6  
  
Sorted array:  
1 3 6 6 9
```

Program -9**Question:**

Implement 0/1 Knapsack problem using dynamic programming.

code :

```
#include<stdio.h>

int max(int a, int b)
{
    if(a > b)
        return a;
    else
        return b;
}

int main()
{
    int n, W;
    int wt[10], val[10];
    int V[10][10];
    int i, j;

    printf("Enter number of items: ");
    scanf("%d", &n);

    printf("Enter weights:\n");
    for(i=1; i<=n; i++)
        scanf("%d", &wt[i]);

    printf("Enter profits:\n");
    for(i=1; i<=n; i++)
```

```

    scanf("%d", &val[i]);
printf("Enter knapsack capacity: ");
scanf("%d", &W);
for(i=0; i<=n; i++)
{
    for(j=0; j<=W; j++)
    {
        if(i==0 || j==0)
        {
            V[i][j] = 0;
        }
        else if(wt[i] > j)
        {
            V[i][j] = V[i-1][j];
        }
        else
        {
            V[i][j] = max(V[i-1][j],
                          V[i-1][j-wt[i]] + val[i]);
        }
    }
}
printf("\nDP Table:\n\n");

for(i=0; i<=n; i++)
{
    for(j=0; j<=W; j++)
    {
        printf("%4d", V[i][j]);
    }
    printf("\n");
}

```

```
printf("\nMaximum Profit = %d\n", V[n][W]);  
return 0;  
}
```

Result :

```
Enter number of items: 4  
Enter weights:  
2 1 3 2  
Enter profits:  
12 10 20 15  
Enter knapsack capacity: 5
```

DP Table:

0	0	0	0	0	0
0	0	12	12	12	12
0	10	12	22	22	22
0	10	12	22	30	32
0	10	15	25	30	37

Leet Code exercise on Knapsack problem using dynamic programming

The screenshot shows a LeetCode submission interface for the problem "Partition Array into Two Equal Sum Subsets". The submission is accepted, with a runtime of 15 ms and a memory usage of 9.23 MB. The code is written in C++ and implements a dynamic programming solution to check if the array can be partitioned into two subsets with equal sums.

Runtime and Memory:

- Runtime: 15 ms
- Memory: 9.23 MB
- Beats: 99.53% (Runtime), 22.72% (Memory)

Code:

```
1 #include <stdbool.h>
2 #include <stdlib.h>
3
4 bool canPartition(int* nums, int numsSize) {
5     int totalSum = 0;
6
7     // 1. Calculate the total sum of the elements
8     for (int i = 0; i < numsSize; i++) {
9         totalSum += nums[i];
10    }
11
12    // If total sum is odd, we cannot partition it into two equal integer subsets
13    if (totalSum % 2 != 0) {
14        return false;
15    }
16
17    int target = totalSum / 2;
18}
```

Testcase:

- Case 1: Input: [1,5,11,5], Output: true
- Case 2: Input: [1,5,11,5], Output: true

More challenges:

- 1981. Minimize the Difference Between Target and Chosen Elements
- 2025. Maximum Number of Ways to Partition an Array
- 2395. Find Subarrays With Equal Sum

Program -10**Question:**

Implement All Pair Shortest paths problem using Floyd's algorithm

code :

```
#include <stdio.h>
#define INF 999
int main()
{
    int n, i, j, k;
    int cost[10][10];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);
        }
    }
    for(k = 0; k < n; k++)
    {
        for(i = 0; i < n; i++)
        {
            for(j = 0; j < n; j++)
            {
                if(cost[i][j] > cost[i][k] + cost[k][j])
                {
                    cost[i][j] = cost[i][k] + cost[k][j];
                }
            }
        }
    }
}
```



```

    }
}
}
printf("Shortest Path Matrix:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
    {
        printf("%d\t", cost[i][j]);
    }
    printf("\n");
}
return 0;
}

```

Result :

```

Enter number of vertices: 4
Enter cost matrix:
0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0
Shortest Path Matrix:
0      5      8      9
999    0      3      4
999    999    0      1
999    999    999    0

```

Leet Code exercise on Floyd's algorithm.

The screenshot displays the LeetCode interface for the problem "Find the Duplicate Number". The submission is accepted, with a runtime of 4 ms and memory usage of 14.84 MB. The code is written in C++ and implements Floyd's cycle-finding algorithm to find the duplicate number in an array.

Runtime Performance:

- Runtime: 4 ms (Beats 61.30%)
- Memory: 14.84 MB (Beats 80.91%)

Code (C++):

```
1 int findDuplicate(int* nums, int numsSize) {
2     // Phase 1: Find the intersection point of the tortoise and hare.
3     int tortoise = nums[0];
4     int hare = nums[0];
5
6     do {
7         tortoise = nums[tortoise]; // Move 1 step
8         hare = nums[nums[hare]]; // Move 2 steps
9     } while (tortoise != hare);
10
11    // Phase 2: Find the entrance to the cycle (the duplicate number).
12    tortoise = nums[0]; // Reset tortoise to start
13    while (tortoise != hare) {
14        tortoise = nums[tortoise]; // Move 1 step
15        hare = nums[hare]; // Move 1 step
16    }
17
18    return hare; // Return the duplicate number
19 }
```

Testcase Results:

- Accepted (Runtime: 0 ms)
- Case 1: Input [1, 3, 4, 2, 2], Output 2, Expected 2

Program -11**Question:**

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

code :

```
#include<stdio.h>
#include<stdbool.h>
#include<string.h>
#define INF 9999999
#define V 5

int G[V][V] = {
    {0, 9, 75, 0, 0},
    {9, 0, 95, 19, 42},
    {75, 95, 0, 51, 66},
    {0, 19, 51, 0, 31},
    {0, 42, 66, 31, 0}};

int main() {
    int no_edge;
    int selected[V];
    memset(selected, false, sizeof(selected));
    no_edge = 0;
    selected[0] = true;
    int x;
    int y;
    printf("Edge : Weight\n");
    while (no_edge < V - 1) {
        int min = INF;
        x = 0;
```

```

y = 0;
for (int i = 0; i < V; i++) {
    if (selected[i]) {
        for (int j = 0; j < V; j++) {
            if (!selected[j] && G[i][j]) {
                if (min > G[i][j]) {
                    min = G[i][j];
                    x = i;
                    y = j;
                }
            }
        }
    }
}
printf("%d - %d : %d\n", x, y, G[x][y]);
selected[y] = true;
no_edge++;
}
return 0;
}

```

Result :

```

Edge : Weight
0 - 1 : 9
1 - 3 : 19
3 - 4 : 31
3 - 2 : 51

```

Question:

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

Code :

```
#include <stdio.h>
#include <stdlib.h>

#define INF 999999

struct Edge {
    int src, dest, weight;
};

struct subset {
    int parent;
    int rank;
};

int find(struct subset subsets[], int i) {
    if (subsets[i].parent != i)
        subsets[i].parent = find(subsets, subsets[i].parent);
    return subsets[i].parent;
}

void Union(struct subset subsets[], int x, int y) {
    int xroot = find(subsets, x);
    int yroot = find(subsets, y);

    if (subsets[xroot].rank < subsets[yroot].rank)
        subsets[xroot].parent = yroot;
```

```

else if (subsets[xroot].rank > subsets[yroot].rank)
    subsets[xroot].parent = xroot;
else {
    subsets[yroot].parent = xroot;
    subsets[xroot].rank++;
}
}

int myComp(const void* a, const void* b) {
    struct Edge* a1 = (struct Edge*)a;
    struct Edge* b1 = (struct Edge*)b;
    return a1->weight - b1->weight;
}

void KruskalMST(int V, int matrix[V][V]) {
    struct Edge* edges = (struct Edge*)malloc(V * V * sizeof(struct Edge));
    int E = 0;

    for (int i = 0; i < V; i++) {
        for (int j = i + 1; j < V; j++) {
            if (matrix[i][j] != 0 && matrix[i][j] != INF) {
                edges[E].src = i;
                edges[E].dest = j;
                edges[E].weight = matrix[i][j];
                E++;
            }
        }
    }

    qsort(edges, E, sizeof(struct Edge), myComp);

    struct Edge result[V];

```

```

int e = 0;
int i = 0;
struct subset* subsets = (struct subset*)malloc(V * sizeof(struct subset));
for (int v = 0; v < V; ++v) {
    subsets[v].parent = v;
    subsets[v].rank = 0;
}
while (e < V - 1 && i < E) {
    struct Edge next_edge = edges[i++];
    int x = find(subsets, next_edge.src);
    int y = find(subsets, next_edge.dest);
    if (x != y) {
        result[e++] = next_edge;
        Union(subsets, x, y);
    }
}
printf("\nEdges in the constructed MST:\n");
int minimumCost = 0;
for (i = 0; i < e; ++i) {
    printf("%d -- %d == %d\n", result[i].src, result[i].dest, result[i].weight);
    minimumCost += result[i].weight;
}
printf("Minimum Cost Spanning Tree: %d\n", minimumCost);
free(subsets);
free(edges);
}

int main() {
    int V;
    printf("Enter number of vertices: ");
    scanf("%d", &V);

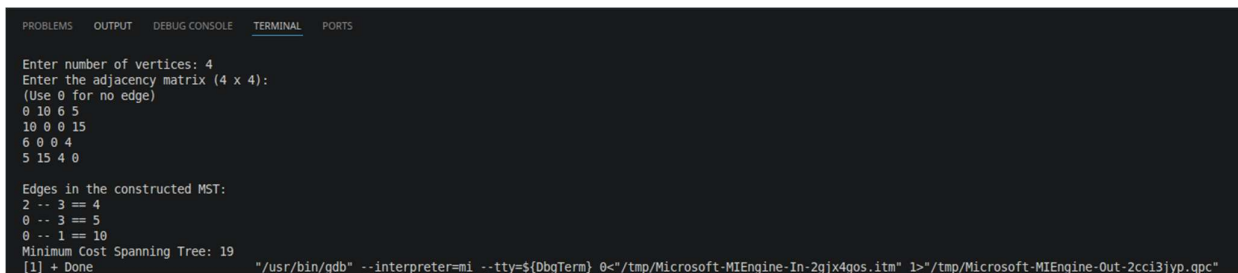
```

```

int matrix[V][V];
printf("Enter the adjacency matrix (%d x %d):\n", V, V);
printf("(Use 0 for no edge)\n");
for (int i = 0; i < V; i++) {
    for (int j = 0; j < V; j++) {
        scanf("%d", &matrix[i][j]);
    }
}
KruskalMST(V, matrix);
return 0;
}

```

Result :



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Enter number of vertices: 4
Enter the adjacency matrix (4 x 4):
(Use 0 for no edge)
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0

Edges in the constructed MST:
2 -- 3 == 4
0 -- 3 == 5
0 -- 1 == 10
Minimum Cost Spanning Tree: 19
[1] + Done
"/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microsoft-MIEngine-In-2gix4qos.itm" 1>"/tmp/Microsoft-MIEngine-Out-2cci3iyp.qpc"

```


Program -12**Question:**

Implement fractional knapsack problem using Greedy technique.

code :

```
#include <stdio.h>

struct Item
{
    int profit;
    int weight;
    float ratio;
};

void sortItems(struct Item item[], int n)
{
    int i, j;
    struct Item temp;

    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(item[i].ratio < item[j].ratio)
            {
                temp = item[i];
                item[i] = item[j];
                item[j] = temp;
            }
        }
    }
}
```

```

}

int main()
{
    int n, i;
    float capacity, totalProfit = 0.0;
    printf("Enter number of items: ");
    scanf("%d", &n);
    struct Item item[n];
    for(i = 0; i < n; i++)
    {
        printf("Enter profit and weight of item %d: ", i + 1);
        scanf("%d%d", &item[i].profit, &item[i].weight);
        item[i].ratio = (float)item[i].profit / item[i].weight;
    }
    printf("Enter capacity of knapsack: ");
    scanf("%f", &capacity);
    sortItems(item, n);
    for(i = 0; i < n; i++)
    {
        if(capacity >= item[i].weight)
        {
            capacity = capacity - item[i].weight;
            totalProfit = totalProfit + item[i].profit;
        }
        else
        {
            totalProfit = totalProfit + (item[i].ratio * capacity);
            break;
        }
    }
}

```

```
printf("Maximum Profit = %.2f\n", totalProfit);  
  
return 0;  
}
```

Result :

```
Enter profit and weight of item 1: 60 10  
Enter profit and weight of item 2: 100 20  
Enter profit and weight of item 3: 120 30  
Enter profit and weight of item 4: 140 40  
Enter profit and weight of item 5: 160 50  
Enter capacity of knapsack: 100  
Maximum Profit = 420.00
```

Program -13**Question:**

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

code :

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define INF 999999

int minDistance(int dist[], bool sptSet[], int V) {
    int min = INF, min_index;

    for (int v = 0; v < V; v++) {
        if (sptSet[v] == false && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}

void printSolution(int dist[], int V, int src) {
    printf("\nVertex   Distance from Source (%d)\n", src);
    for (int i = 0; i < V; i++) {
        if (dist[i] == INF) {
            printf("%d \t\t INF\n", i);
        } else {
            printf("%d \t\t %d\n", i, dist[i]);
        }
    }
}
```

```

    }
}
}

void dijkstra(int V, int graph[V][V], int src) {
    int dist[V];
    bool sptSet[V];

    for (int i = 0; i < V; i++) {
        dist[i] = INF;
        sptSet[i] = false;
    }
    dist[src] = 0;
    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, sptSet, V);
        sptSet[u] = true;
        for (int v = 0; v < V; v++) {
            if (!sptSet[v] && graph[u][v] && dist[u] != INF && dist[u] + graph[u][v] <
dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    printSolution(dist, V, src);
}

int main() {
    int V, src;
    printf("Enter number of vertices: ");
    scanf("%d", &V);
    int graph[V][V];
    printf("Enter the adjacency matrix (%d x %d):\n", V, V);
    printf("(Use 0 for no edge/self-loops)\n");
}

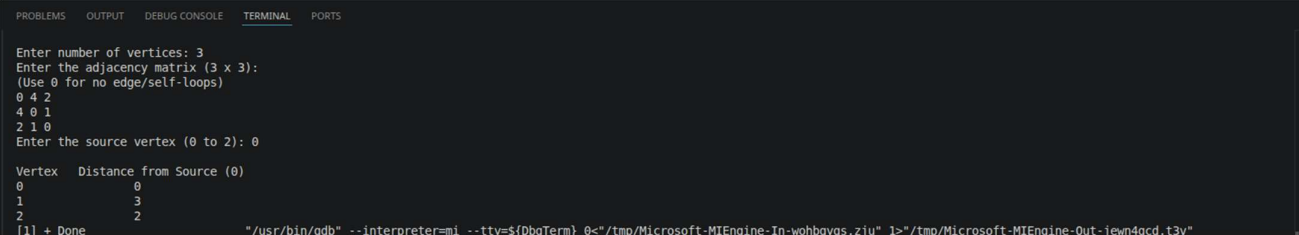
```

```

for (int i = 0; i < V; i++) {
    for (int j = 0; j < V; j++) {
        scanf("%d", &graph[i][j]);
    }
}
printf("Enter the source vertex (0 to %d): ", V - 1);
scanf("%d", &src);
dijkstra(V, graph, src);
return 0;
}

```

Result :



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Enter number of vertices: 3
Enter the adjacency matrix (3 x 3):
(Use 0 for no edge/self-loops)
0 4 2
4 0 1
2 1 0
Enter the source vertex (0 to 2): 0
Vertex  Distance from Source (0)
0       0
1       3
2       2
[1] + Done
"/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microsoft-MIEngine-In-wohbayqs.zju" 1>"/tmp/Microsoft-MIEngine-Out-jewn4qcd.t3v"

```

Program -14**Question:**

Implement “N-Queens Problem” using Backtracking.

code :

```
#include<stdio.h>
#include<stdlib.h>
int x[20], count = 0;
int place(int k, int i)
{
    int j;
    for(j = 1; j < k; j++)
    {
        if((x[j] == i) || (abs(x[j] - i) == abs(j - k)))
            return 0;
    }
    return 1;
}
void nqueen(int k, int n)
{
    int i, j;
    for(i = 1; i <= n; i++)
    {
        if(place(k, i))
        {
            x[k] = i;
            if(k == n)
            {
                count++;
                printf("\nSolution %d:\n\n", count);
            }
        }
    }
}
```

```

        for(i = 1; i <= n; i++)
        {
            for(j = 1; j <= n; j++)
            {
                if(x[i] == j)
                    printf(" Q ");
                else
                    printf(" . ");
            }
            printf("\n");
        }
    }
else
{
    nqueen(k + 1, n);
}
}
}

int main()
{
    int n;
    printf("Enter number of queens: ");
    scanf("%d", &n);
    nqueen(1, n);
    if(count == 0)
        printf("\nNo solution exists");
    return 0;
}

```


Result :

```
Enter number of queens: 4

Solution 1:

. Q . .
. . . Q
Q . . .
. . Q .

Solution 2:

. . Q .
Q . . .
. . . Q
. Q . .
```